# Ders 24 Çalışma Özeti

## Ders 24 Çalışma Özeti

### Genel Konular

- Dersin tarihi 14 Aralık 2021 Salı. Bu kayıt bir uygulama (laboratuvar) oturumudur.
  - Oturumu dersin asistanı anlatıyor. Slayt başlığı "İşletim Sistemleri Uygulama".
  - Asıl anlatım yaklaşık 00:06'da başlar. Öncesi bekleme ve sohbettir.
- Laboratuvar takvimi
  - Bir önceki uygulama teknik sorun yüzünden tekrarlandı, laboratuvar bir hafta kaydı.
  - Bu hafta kernel modülü yerine semaforlar işlendi. Ödev ve kernel modülü laboratuvarı için dersin hocasına (Cihan Hoca) sorulması önerildi.
- Senkronizasyon ihtiyacı, açlık (starvation) ve ölümcül kilitlenme (deadlock)
- POSIX semaforları: isimli (named) ve isimsiz (unnamed)
- POSIX mesaj kuyrukları (message queues)
- POSIX paylaşılan bellek (shared memory)
- POSIX iş parçacıkları (pthreads): oluşturma, sonlandırma, `join`, `detach`
- Muteksler (mutex) ve koşul değişkenleri (condition variables)

### Hocanın Özellikle Vurguladığı Kısımlar

- Paylaşılan bir kaynağa aynı anda erişen süreçler programın sonucunu öngörülemez yapar.
  - Amaç, aynı girdiyle her seferinde aynı sonucu almaktır. "1 + 1 bazen 2, bazen 3" kabul edilemez.
- Paralel programlarda hata bulmak zordur.
  - Derleyici hata vermez. Sorun yalnız çalışma sırasına bağlı olarak ortaya çıkar.
- Mesaj kuyruğunda boş kuyruğu sürekli yoklamak (busy waiting) işlemciyi meşgul eder.
  - Bunun yerine `mq_notify` ile mesaj gelince uyarı almak tercih edilir.
- Paylaşılan bellek, mesaj kuyruğundan hızlıdır.
  - Sistem çağrısı yalnız alanı oluştururken kullanılır. Sonraki erişimler normal bellek erişimidir.
- Örnek kodların çıktıları slaytlarda var. İsteyen Ubuntu'da kendisi deneyebilir.

### Kısa Tekrar Notları

- Açlık: süreç beklediği kaynağa hiçbir zaman erişemez.
- Deadlock: her süreç bir sonrakinin beklediği kaynağı tutar. Kimse bırakmadığı için hepsi sonsuza kadar bekler.
- Semafor = tam sayı değer + bekleyen süreçlerin kuyruğu.
  - `sem_wait`: değer 0'dan büyükse 1 azaltır; 0 ise değer artana kadar bekler.
  - `sem_post`: değeri 1 artırır, bekleyen varsa birini uyandırır.
- İsimli semafor: `sem_open`, `sem_wait`, `sem_post`, `sem_close`, `sem_unlink`.
- İsimsiz semafor: `sem_init`, `sem_wait`, `sem_post`, `sem_destroy`.
- Mesaj kuyruğu: `mq_open`, `mq_send`, `mq_receive`, `mq_close`, `mq_unlink`, `mq_notify`, `mq_getattr`/`mq_setattr`, `mq_timedsend`/`mq_timedreceive`.
  - `mq_receive` en yüksek öncelikli mesajlardan en eskisini alır.
- Paylaşılan bellek: `shm_open`, `ftruncate`, `mmap`, `munmap`, `shm_unlink`.
- Thread kendi yığınına (stack), yazmaçlarına (registers), program sayacına (program counter) ve kimliğine sahiptir. Bellek, açık dosyalar ve sinyal işleyicileri paylaşılır.
- `pthread_create` başarılıysa 0 döndürür. `pthread_join` thread'in bitmesini bekler.

- Mutex: `pthread_mutex_lock` / `pthread_mutex_unlock`. Koşul değişkeni: meşgul beklemeden, koşul sağlanınca uyandırma.

## Detaylı Açıklamalar

### Senkronizasyon neden gerekli?

Dört süreç aynı `counter` değişkenini değiştirmek istesin. P2 değeri 2, P4 değeri 1 artırıyor. İkisi aynı eski değeri okuyup üzerine yazarsa sonuç 3 yerine yalnız 1 ya da 2 artmış olabilir. Bu bir yarış durumudur (race condition). Semafor gibi yapılar bu kaynağa erişimi sıraya koyar.

Kaynak paylaştırırken iki durumdan kaçınılır:

- Açlık: süreç kaynağı bekler ama kaynak ona hiç gelmez.
- Deadlock: kavşakta birbirinin yolunu kesen dört araba gibi. P1'in beklediğini P2, P2'ninkini P3, P3'ünkünü P4, P4'ünkünü P1 tutar.

### Semaforlar

Semaforu bir anahtar kutusu gibi düşünün. Değer, kutudaki anahtar sayısıdır. Anahtar varsa süreç kritik bölgeye (critical section) girer ve değer 1 azalır. Anahtar yoksa süreç bekler. İşi bitince anahtarı geri koyar ve değer 1 artar.

Derste gösterilen örnekte isimli semafor `sem_open` ile açılır ve değeri 1 yapılır. A ve B süreçleri on kez kritik bölgeye girip çıkar ve saatleri yazdırır. Çıktıda iki sürecin kritik bölgede hiçbir zaman aynı saniyede bulunmadığı görülür.

### Mesaj kuyrukları

- Kalıcı (persistent) nesnelerdir. Oluşturulmaları ve işleri bitince silinmeleri gerekir.
- En fazla mesaj sayısı ve en büyük mesaj boyu oluşturulurken belirlenir.
- İsim `/` ile başlamalı ve başka `/` içermemelidir.
- `mq_attr` yapısının 4 alanı vardır: `mq_flags`, `mq_maxmsg`, `mq_msgsize`, `mq_curmsgs`.
- `O_EXCL`: kuyruk zaten varsa `mq_open` hata verir. `O_NONBLOCK`: çağrı bloklanmaz.
- Okuyan ve yazan sürecin aynı anda açık olması gerekmez. Yazar mesaj bırakıp kuyruğu kapatabilir. Okuyucu sonra açıp mesajları okuyabilir.
- `mq_send`: mesaj boyu kuyruğun mesaj boyundan büyük olamaz. Kuyruk doluysa ve `O_NONBLOCK` yoksa çağrı yer açılana kadar bekler.
- `mq_receive`: Derste gösterilen örnekte üç mesaj gönderildi; öncelikleri 11, 110 ve 17'ydi. Alıcı önce önceliği 110 olan mesajı aldı.
- `O_NONBLOCK` ile boş kuyruktan okuma `-1` döndürür. Ama bunu döngüde denemek meşgul beklemedir.
- `mq_notify`: kuyruğa mesaj gelince kayıtlı süreç bir `sigevent` ile uyarılır.
  - Bildirim bir kez gönderilir, sonra kayıt silinir. Tekrar bildirim için yeniden kayıt olmak gerekir.
  - Uyarılacak süreci değiştirmek için önce `mq_notify(mqdes, NULL)` ile kayıt kaldırılır.

### Paylaşılan bellek

Mesaj kuyruğunda her gönderme ve alma bir sistem çağrısıdır. Paylaşılan bellekte ise süreçler aynı bellek bölgesine doğrudan erişir. Bu yüzden daha hızlıdır. Ama senkronizasyonu programcı sağlamalıdır.

- `shm_open`: nesneyi oluşturur ya da açar. Hata durumunda `-1` döner.
  - `O_CREAT`: gerekirse oluştur. `O_EXCL`: varsa hata ver. `O_TRUNC`: varsa boyunu 0 yap. `O_RDWR` / `O_RDONLY`: erişim kipi.
- `ftruncate`: nesnenin boyunu ayarlar. Fazlalık atılır, eksik kısım sıfır baytlarla doldurulur.
- `mmap`: nesnenin bir bölgesini sürecin adres alanına eşler. `addr` genelde `NULL` (0) verilir; yeri işletim sistemi seçer. Eşlenen adresi döndürür.
- `munmap`: eşlemeyi kaldırır.
- `shm_unlink`: nesnenin adını siler. Nesne, onu kullanan son süreç eşlemeyi bırakınca yok olur.

### Thread'ler

`fork` yeni bir süreç oluşturur ve sürecin adres alanını kopyalar. Bu maliyetlidir. Aynı süreç içinde yeni bir thread oluşturmak daha ucuzdur. Thread'ler süreç belleğini, açık dosyaları ve sinyal işleyicilerini paylaşır. Her thread'in kendi yığını, yazmaçları, program sayacı ve kimliği vardır.

```
int pthread_create(pthread_t *tid, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
```

- Thread fonksiyonuna tek bir argüman verilir. Birden fazla değer göndermek için bir `struct` kullanılır.
- `main` de bir thread'tir. Oluşturduğu thread'leri `pthread_join` ile bekler.
- Thread fonksiyonundan dönünce ya da `pthread_exit` çağrılınca thread sonlanır.
- Joinable thread'ler `join` edilebilir. Detached thread'ler edilemez; `pthread_attr_setdetachstate` ile ayarlanır.

Derste gösterilen dört thread'li örnek:

- Thread 1 metnin uzunluğunu sayar. Sonuç 36.
- Thread 2 boşlukları sayar. Sonuç 5.
- Thread 3 metni büyük harfe çevirir. Thread 4 küçük harfe çevirir.

Sayma sonuçları her çalıştırmada aynıdır. Ama büyük ve küçük harf thread'leri aynı metin üzerinde yarışır. Her çalıştırmada metnin farklı bir kısmı büyük, farklı bir kısmı küçük çıkar. Bu, senkronizasyonsuz ortak veri erişiminin öngörülemezliğini gösterir.

İkinci adımda dört fonksiyon bir fonksiyon işaretçisi dizisine konur. Böylece thread'ler tek bir döngüyle oluşturulur.

### Muteks ve koşul değişkenleri

Karşılıklı dışlama (mutual exclusion), tuvalete girince kapıyı kilitlemeye benzer. İçerideyken kimse giremez, çıkınca kilit açılır. `pthread_mutex_lock` kilidi alır; kilit başkasındaysa bekler. `pthread_mutex_unlock` kilidi bırakır.

Kapının açılıp açılmadığını sürekli denemek meşgul beklemedir. Koşul değişkeni, "kapı açılınca zil çalsın" demek gibidir. Thread `pthread_cond_wait` ile uyur. Koşul sağlanınca başka bir thread `pthread_cond_signal` ile onu uyandırır.

### Ders sonrası notlar

- `sem_close` ve `sem_unlink` derste ters anlatıldı.
  - `sem_close`: yalnız bu sürecin isimli semaforla bağlantısını kapatır.
  - `sem_unlink`: semaforun adını sistemden siler. Semafor, açık tutan son süreç de kapatınca yok olur.
- Derste "fork ile oluşturulan alt süreç açık mesaj kuyruğunu devralamaz" dendi. POSIX ve Linux'ta bu yanlış: alt süreç, ebeveynin açık mesaj kuyruğu tanımlayıcılarının kopyalarını devralır (`man 2 fork`). Derlenerek denendi: `fork` sonrası alt süreç, ebeveynin tanımlayıcısıyla `mq_getattr` çağırabildi ve kuyrukta 3 mesaj gördü. Ayrıca `fork` thread değil, süreç oluşturur.
- `mq_close` ve `mq_unlink` için "kuyruk, içindeki mesajlar tükenene kadar silinmez" dendi. Doğrusu:
  - `mq_close` kuyruğu hiç silmez, yalnız tanımlayıcıyı kapatır.
  - `mq_unlink` adı hemen siler. Kuyruğun kendisi, onu açık tutan bütün süreçler kapatınca yok olur. Mesajların okunup okunmamasıyla ilgisi yoktur. Derlenerek denendi: içinde mesaj varken `mq_unlink` sonrası kuyruk adla yeniden açılamadı, ama açık tanımlayıcıdan mesaj okunabildi.
- Derste `munmap` için "çok kullanılmaz, yerine `shm_unlink` geliştirildi" dendi. İkisi farklı işler yapar ve birbirinin yerine geçmez. `munmap` eşlemeyi kaldırır, `shm_unlink` nesnenin adını siler. Derlenerek denendi: `shm_unlink` sonrası eşlenmiş bellekteki veri hâlâ okunabildi, ama `shm_open` ile aynı ad açılamadı.
- `pthread_create` başarılı olunca 0 döndürür; derste "sıfır değil" dendi. Hata durumunda sıfırdan farklı bir hata kodu döner. Derlenerek denendi: dönüş değeri 0 çıktı.

- Thread fonksiyonu `void` değil, `void *` döndürmelidir.
- “Struct parametre yerel değişken olamaz” kuralı tam doğru değil. Asıl şart, yapının thread onu kullandığı sürece geçerli kalmasıdır. `main` içinde tanımlanıp `join` ile beklenirse yerel değişken de kullanılabilir. Derlenerek denendi: `main` içindeki yerel `struct foo {3, 4}` thread’e verildi ve thread `x=3 y=4` yazdı.
- “Mutex değişkenleri statik olmalı” ifadesi yalnız `PTHREAD_MUTEX_INITIALIZER` ile ilklendirme için geçerlidir. Dinamik muteksler `pthread_mutex_init` ile ilklendirilir.
- POSIX semaforunun değeri gerçekten negatif olamaz. Ders kitabındaki (Silberschatz) semafor gerçeklemesinde ise değer negatif olabilir. Negatif değerin mutlak değeri bekleyen süreç sayısını verir.
- Modern sistemlerde `fork` belleği hemen kopyalamaz; yazmada kopyalama (copy-on-write) kullanır. Yine de thread oluşturmaktan daha maliyetlidir.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.